

K K SIngh

Machine Learning

E3S2 2025-26

January 4, 2026

Springer Nature

Contents

0.1	Definition of Machine Learning	1
0.2	Choosing the Training Experience (E)	1
0.2.1	Conceptual System Modules	2
0.3	Target Function, Representation, and Learning Algorithm	2
0.3.1	Target Function (V) and Hypothesis Space (H)	2
0.3.2	Learning Algorithm: Minimizing Error via Gradient Descent	3
0.4	Types of Machine Learning	4
0.4.1	Supervised Learning	4
0.4.2	Unsupervised Learning	4
0.4.3	Semi-Supervised Learning	5
0.4.4	Reinforcement Learning	5
0.4.5	Comparison Summary	5
0.4.6	An Example of of Supervised Learning	6
0.5	An example of unsupervised learning: with K-Means Algorithm for Anomaly Detection	7
0.5.1	Example Dataset and Features	8
0.5.2	Core Components and Parameters	8
0.6	K-Means Step-by-Step Walkthrough ($K = 2$)	8
0.6.1	Step 1: Initialization	8
0.6.2	Step 2: Assignment Step (Iteration 1)	8
0.6.3	Step 3: Update Step (Iteration 1)	9
0.6.4	Visualization of the Process	9
0.7	Fraud Detection using the Final Model	9
0.8	Reinforcement Learning (RL) Overview	9
0.8.1	The Environment and Setup	9
0.8.2	The Q-Learning Mechanism	10
0.8.3	Q-Table Progression	10
0.9	Concept Learning, Version Space, and the Candidate Elimination Algorithm	13
0.9.1	1. Target Concept (c)	13
0.9.2	Worked Example: Candidate Elimination Algorithm	15
0.10	Inductive Bias	19
0.10.1	Definition	20
0.10.2	Need for Inductive Bias	20

0.10.3	Example: Inductive Bias in Concept Learning	20
0.10.4	Formal View	20
0.10.5	Illustrative Diagram	21
0.11	Types of Inductive Bias	21
0.12	Comparison of Algorithm Biases	21
0.13	Summary	22
0.14	Decision Tree Induction Method	23
0.15	The Dataset: PlayTennis	24
0.16	Mathematical Foundations	24
0.16.1	Entropy	24
0.16.2	Information Gain	24
0.17	Level-Wise Construction	25
0.17.1	Level 0: Root Node Computation	25
0.17.2	Level 1: Branching	25
0.17.3	Sub-Table: Outlook = Sunny	25
0.18	Decision Tree Rules	25
0.19	Inductive Bias	26
0.20	Unit-2	27
0.21	Artificial Neural Networks	28
0.22	Biological vs Artificial Neural Networks	28
0.23	Artificial Neuron Model	28
0.24	Activation Functions	28
0.24.1	Step Function	29
0.24.2	Sign Function	29
0.24.3	Sigmoid Function	30
0.24.4	Forward Pass Computation	31
0.24.5	Error Function	32
0.24.6	Backpropagation as Chain Rule of Derivatives	32
0.24.7	Backpropagation Algorithm Summary	33
0.25	Unit-2.1	35
0.26	Gradient-Based Optimization Methods	36
0.26.1	Problem Formulation	36
0.26.2	Gradient Descent: Vector–Matrix Form	36
0.26.3	Stochastic Gradient Descent (SGD)	36
0.26.4	Geometric Interpretation	37
0.26.5	Momentum-Based Gradient Descent	37
0.26.6	RMSProp	37
0.26.7	Adam Optimizer	38
0.26.8	Comparison of Optimization Methods	38
0.26.9	Summary	38

0.1 Definition of Machine Learning

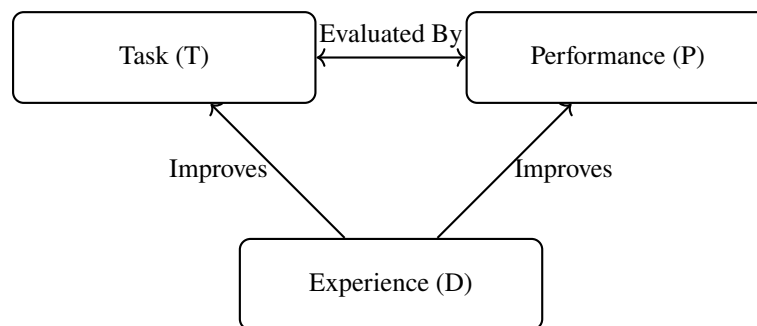
Machine Learning is a branch of Artificial Intelligence concerned with the design and development of algorithms that allow computers to evolve behaviors based on empirical data.

According to the formal learning paradigm:

Machine Learning is the study of algorithms that improve their performance (P) at some task (T) with experience (D).

Thus, Machine Learning always involves the following three components:

- **Task (T):** The objective the system is trying to achieve (e.g., classification, prediction, detection).
- **Performance Measure (P):** A metric that evaluates how well the system performs the task (e.g. accuracy, error rate).
- **Experience (D):** The dataset or past interaction from which the system learns and improves.



In summary, for any system to be considered a Machine Learning system, it must:

1. have a well-defined **task (T)**,
2. have a measurable **performance criterion (P)**, and
3. have access to some **experience or dataset (E)** that helps it improve.

0.2 Choosing the Training Experience (E)

The characteristics of the training experience are critical for system success:

1. **Direct vs. Indirect Feedback:** Training can provide direct feedback (e.g., individual board states + correct move) or indirect feedback via the final outcome (win/loss). Indirect feedback leads to the Credit Assignment Problem, where it is difficult to determine how much credit or blame each intermediate move deserves for the final outcome
 2. **Learner Control:** training examples can be selected by a teacher, queried by the learner, or generated by the learner itself (self-play).
 3. **Match of Distributions:** Learning is most reliable when the training distribution matches the future test distribution. However, training via self-play may result in a mismatched distribution, which can limit generalization.
-

0.2.1 Conceptual System Modules

The complete learning system is made up of four interacting components:

1. **Performance System:** Uses the current learned function f to execute moves during a game.
 2. **Critic:** Generates training examples $\langle b, V_{\text{train}}(b) \rangle$ by estimating the true target value for non-final board states encountered during training.
 3. **Generalizer:** Implements the LMS algorithm to generalize from the training examples and output a new, improved set of weights (the hypothesis function f).
 4. **Experiment Generator:** Manages the overall training process, which in this case involves generating new board states for exploration via self-play.
-

0.3 Target Function, Representation, and Learning Algorithm

The core of the machine learning system design is the process of defining the unknown optimal function and devising a method to approximate it from experience.

0.3.1 Target Function (V) and Hypothesis Space (H)

The program's ambitious task of playing checkers (**T**) is simplified by decomposing it into the problem of learning an **Evaluation Function** (**V**). This function maps any board state b to a numerical value, estimating its quality:

$$\mathbf{V} : \text{Board} \rightarrow \mathbb{R}$$

The ideal, but unknown, target function $\mathbf{V}(\mathbf{b})$ is defined by the eventual game outcome:

$$\mathbf{V}(\mathbf{b}) = \begin{cases} +100 & \text{if } b \text{ is a final board state won by the program} \\ -100 & \text{if } b \text{ is a final board state lost by the program} \\ 0 & \text{if } b \text{ is a final board state drawn} \\ \text{An estimate of the true value} & \text{for any non-final board state} \end{cases}$$

The learner must search for a function f that approximates $\mathbf{V}$. This function f , called the **hypothesis**, is constrained to a specific **Hypothesis Space** (**H**). For simplicity, f is chosen to be a **linear combination** of six key features $(x_1, \dots, x_6)$ of the board state:

$$\mathbf{f}(\mathbf{b}) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6$$

The features x_i represent measurable board properties (e.g., number of pieces, number of kings), and the learning task is reduced to finding the optimal numerical coefficients or **weights** ($\mathbf{w}_i$).

0.3.2 Learning Algorithm: Minimizing Error via Gradient Descent

The learning algorithm's goal is to find the set of weights $\mathbf{W} = \langle w_0, \dots, w_6 \rangle$ that minimizes the error between the function's output $f(b)$ and the training value $V_{\text{train}}(b)$ for all observed examples.

1. Error Function (Least Squares Method)

The standard approach is to minimize the **squared error** over the entire training set D :

$$E(\mathbf{W}) \equiv \frac{1}{2} \sum_{\langle b, V_{\text{train}}(b) \rangle \in D} [V_{\text{train}}(b) - f(b)]^2$$

This is the objective function used in the **Least Squares Method**. Finding the weights $\mathbf{W}$ that globally minimize this error is the core computational challenge.

2. Gradient Descent

The method used to minimize $E(\mathbf{W})$ is **Gradient Descent**. This technique starts with an arbitrary set of weights and iteratively modifies them in the direction of the steepest descent on the error surface defined by $E(\mathbf{W})$. The direction of steepest descent is given by the negative of the **gradient** of $E(\mathbf{W})$:

$$\nabla E(\mathbf{W}) = \left\langle \frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_6} \right\rangle$$

The update rule for all weights simultaneously is:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla E(\mathbf{W})$$

where η is the **learning rate** (a small positive constant controlling the step size).

3. The Delta Rule (LMS)

The **Delta Rule** (or Widrow-Hoff rule, which for this application is equivalent to the **Least Mean Squares (LMS) rule**) is a variant of gradient descent applied to a single training example at a time. It updates the weights incrementally for each training example $\langle b, V_{\text{train}}(b) \rangle$:

The instantaneous error for a single example is $E_{\text{instantaneous}}(b) = \frac{1}{2} [V_{\text{train}}(b) - f(b)]^2$. The partial derivative of this error with respect to a single weight w_i is:

$$\frac{\partial E_{\text{instantaneous}}}{\partial w_i} = -[V_{\text{train}}(b) - f(b)]x_i$$

Substituting this into the gradient descent update ($\mathbf{w}_i \leftarrow \mathbf{w}_i - \eta \frac{\partial E}{\partial \mathbf{w}_i}$) yields the final **Delta Rule/LMS Weight Update Rule**:

$$\mathbf{w}_i \leftarrow \mathbf{w}_i + \eta [V_{\text{train}}(\mathbf{b}) - f(\mathbf{b})] \mathbf{x}_i$$

This rule allows the system to learn iteratively, adjusting the weights to reduce the error on the current training example, gradually converging toward a hypothesis function f that approximates the ideal target function $\mathbf{V}$.

0.4 Types of Machine Learning

Machine Learning can be broadly classified into four major categories: **Supervised Learning**, **Unsupervised Learning**, **Semi-Supervised Learning**, and **Reinforcement Learning**. Each of these learning paradigms differs in the type of data available and how the learning algorithm updates itself.

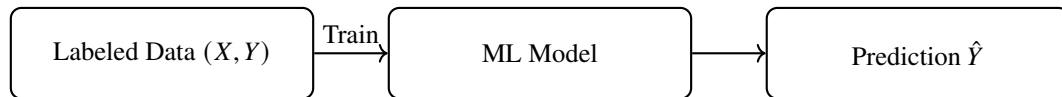
0.4.1 Supervised Learning

Supervised learning involves training a model using labeled data. Each input example has a corresponding output (label). The goal is to learn a mapping function:

$$f(X) \rightarrow Y$$

Examples

- Predicting house prices
- Image classification



0.4.2 Unsupervised Learning

Unsupervised learning uses unlabeled data and discovers hidden patterns. There is no target value; the algorithm tries to structure the data.

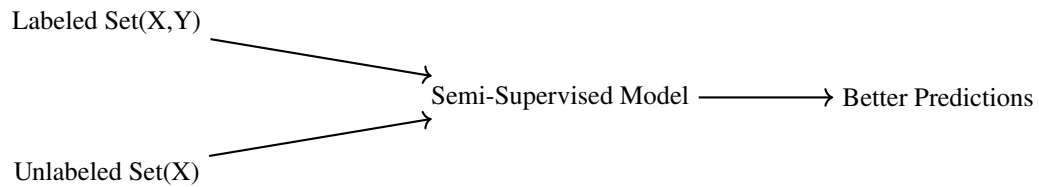
Examples

- Clustering customers
- Finding groups in data
- Dimensionality reduction (PCA)



0.4.3 Semi-Supervised Learning

Semi-supervised learning uses a small amount of labeled data combined with a large amount of unlabeled data. The model benefits from labeled data but also learns structure from unlabeled data.

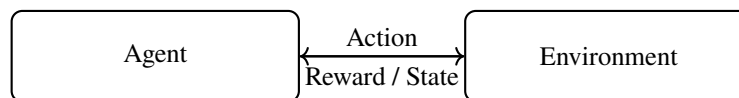


Examples

- Speech recognition
- Medical image classification (few labels)

0.4.4 Reinforcement Learning

Reinforcement learning involves an **agent** learning by interacting with an **environment**. The agent takes actions and receives rewards. The goal is to maximize cumulative reward.



Examples

- Game playing (Chess, Checkerboard)
- Robot navigation
- Self-driving cars

0.4.5 Comparison Summary

Learning Type	Data Type	Goal	Examples
Supervised	Labeled data	Predict output	Classification, Regression
Unsupervised	Unlabeled data	Discover structure	Clustering, PCA
Semi-Supervised	Few labels + many unlabeled	Improve accuracy	Speech, Medical imaging
Reinforcement	Rewards from environment	Maximize reward	Games, Robotics

0.4.6 An Example of of Supervised Learning

We aim to find the optimal parameters (m, b) for a linear model $\hat{y} = mx + b$ using Gradient Descent (GD) on the following small dataset ($n = 3$):

Temperature (x)	10	20	30
Consumption (y)	15	25	35

We use a **Learning Rate** $\alpha = 0.01$ and start with **Initial Parameters** $m_0 = 0$ and $b_0 = 0$.

0.4.6.1 Cost Function and Gradients

The **Mean Squared Error (MSE) Cost Function** $J(m, b)$ to be minimized is:

$$J(m, b) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = \frac{1}{2n} \sum_{i=1}^n (mx_i + b - y_i)^2$$

The **Update Rules** for Gradient Descent are:

$$m_{new} = m_{old} - \alpha \frac{\partial J}{\partial m} \quad \text{and} \quad b_{new} = b_{old} - \alpha \frac{\partial J}{\partial b}$$

where the gradients are:

$$\frac{\partial J}{\partial m} = \frac{1}{n} \sum_{i=1}^n (mx_i + b - y_i)x_i$$

$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (mx_i + b - y_i)$$

Gradient Descent Iterative Computation

Data: (10, 15), (20, 25), (30, 35). **Learning Rate** (α): 0.01.

i	m (Start)	b (Start)	$\frac{\partial J}{\partial m}$	$\frac{\partial J}{\partial b}$	Δm	Δb	m (End)	b (End)
1	0.0000	0.0000	-566.67	-25.00	5.67	0.25	5.67	0.25
2	5.6700	0.2500	2082.78	88.58	-20.83	-0.89	-15.16	-0.64
3	-15.1600	-0.6400	-7654.57	-328.86	76.55	3.29	61.39	2.65

Note on Divergence: The parameters (m, b) are oscillating dramatically and increasing in magnitude, indicated by the alternating large positive and negative gradients. This confirms that the learning rate $\alpha = 0.01$ is too large for this problem, causing the optimization process to diverge from the solution.

Visualizations

The following plot shows the initial line ($m = 0, b = 0$) and the line after the first step of optimization ($m = 5.67, b = 0.25$), relative to the true data points.

This diagram conceptually illustrates how GD takes steps toward the minimum of the convex cost function $J(\theta)$ (where θ represents the parameters m and b).

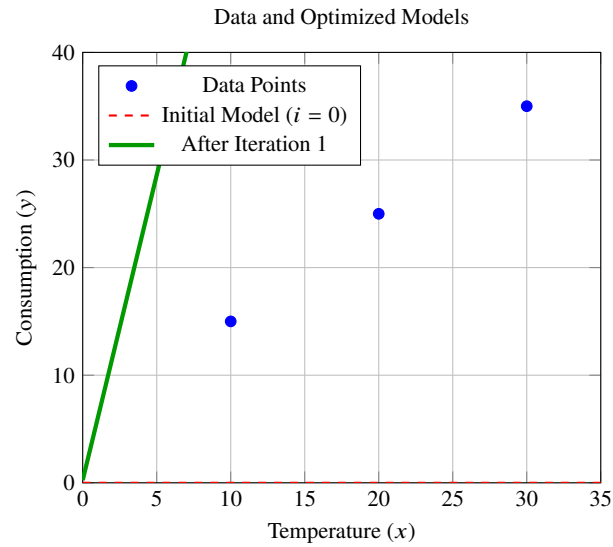


Fig. 0.1 Visualization of the data and the linear model's progression after 1 iteration.

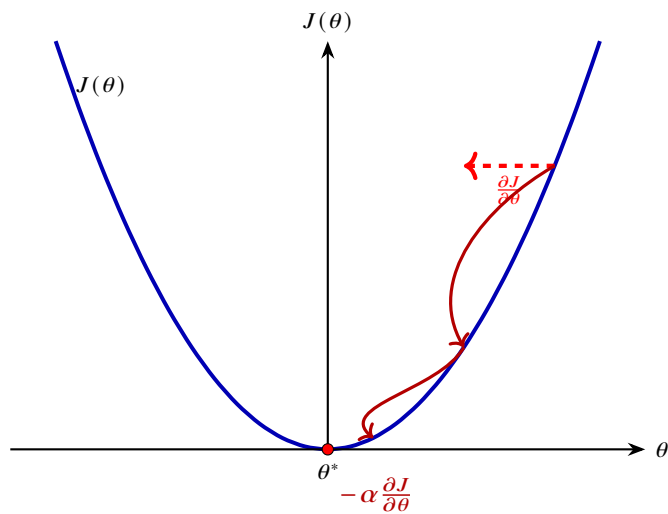


Fig. 0.2 Conceptual diagram of Gradient Descent: taking steps proportional to the negative gradient to reach the minimum cost.

0.5 An example of unsupervised learning: with K-Means Algorithm for Anomaly Detection

In unsupervised fraud detection, K-Means groups transactions based on shared features, defining "normal" spending profiles. Transactions that fall far from any established cluster center (centroid) are flagged as potential outliers/fraud.

0.5.1 Example Dataset and Features

We use a dataset $\mathcal{D}$ with five samples ($N = 5$), simplified to two key features: Transaction Amount (A) and Time Since Last Transaction (T).

	Sample (x)	Amount (A)	Time Since Last (T)	Type
Example Transaction Dataset ($N = 5$)	x_1	100	20	High-Value, Fast
	x_2	95	22	High-Value, Fast
	x_3	5	180	Low-Value, Slow
	x_4	110	18	High-Value, Fast
	x_5	15	170	Low-Value, Slow

0.5.2 Core Components and Parameters

- **Number of Clusters (K):** $K = 2$ (representing distinct spending patterns).
- **Distance Metric:** Euclidean Distance, $d(\mathbf{x}, \mu)$.
- **Centroids (μ):** The mean vector for each cluster.

0.6 K-Means Step-by-Step Walkthrough ($K = 2$)

0.6.1 Step 1: Initialization

We randomly select x_1 and x_3 as the initial cluster centroids:

- $\mu_1 = (100, 20)$
- $\mu_2 = (5, 180)$

0.6.2 Step 2: Assignment Step (Iteration 1)

We assign each point to the nearest centroid, minimizing the Euclidean distance:

$$\text{Assign } x \text{ to } \arg \min_j d(x, \mu_j)$$

	Sample	$d(x, \mu_1)$	$d(x, \mu_2)$	Assignment	Cluster
Distance Calculation and Assignment	$x_1(100, 20)$	0.0	160.7	μ_1	C_1
	$x_2(95, 22)$	5.4	158.8	μ_1	C_1
	$x_3(5, 180)$	158.8	0.0	μ_2	C_2
	$x_4(110, 18)$	10.2	168.0	μ_1	C_1
	$x_5(15, 170)$	149.3	11.2	μ_2	C_2

The resulting clusters are:

- $C_1 = \{x_1, x_2, x_4\}$ (High-Value, Fast transactions)
- $C_2 = \{x_3, x_5\}$ (Low-Value, Slow transactions)

0.6.3 Step 3: Update Step (Iteration 1)

We recalculate the centroids as the mean of all points in the new clusters:

$$\mu'_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$$

- $\mu'_1 = \left(\frac{100+95+110}{3}, \frac{20+22+18}{3} \right) = (101.67, 20.0)$
- $\mu'_2 = \left(\frac{5+15}{2}, \frac{180+170}{2} \right) = (10.0, 175.0)$

0.6.4 Visualization of the Process

The diagram below shows the initial data points, the randomly chosen initial centroids (μ_1, μ_2), and the final calculated centroids (μ'_1, μ'_2) after one iteration.

0.7 Fraud Detection using the Final Model

After the K-Means algorithm converges (i.e., the centroids no longer move), we use the final clusters for anomaly detection:

1. **Define Anomaly Score:** For a new transaction $\mathbf{x}_{\text{new}}$, the anomaly score is the distance to its assigned cluster centroid μ_j .

$$\text{Anomaly Score} = d(\mathbf{x}_{\text{new}}, \mu_j)$$

2. **Set Threshold:** A statistical threshold (e.g., based on the 95th percentile of distances in the training data) is established.
3. **Flag Fraud:** If Anomaly Score > Threshold, the transaction is flagged. For instance, a transaction $\mathbf{x}_{\text{fraud}} = (5000, 100)$ would be assigned to C_1 but would have an extremely large distance to $\mu'_1 = (101.67, 20.0)$, indicating anomalous behavior.

0.8 Reinforcement Learning (RL) Overview

Reinforcement Learning is an area of Machine Learning where an **Agent** (e.g., a robot) learns to make decisions by interacting with an **Environment** (e.g., a grid map). The Agent's goal is to maximize a cumulative **Reward**.

The core algorithm used here is **Q-Learning**, a popular model-free technique that finds the optimal set of actions for an agent in any given state.

0.8.1 The Environment and Setup

The Grid Map Environment

A simple 4×4 grid where each cell is a distinct state (S_1 to S_{16}).

State	1	2	3	4
A	S_1	S_2	S_3	S_4 (Goal)
B	S_5	S_6	S_7	S_8 (Wall)
C	S_9	S_{10}	S_{11}	S_{12}
D	S_{13} (Start)	S_{14}	S_{15}	S_{16}

- **Actions (A):** North (N), South (S), East (E), West (W).

The Reward System

Action Result	Reward (R)
Move into the Goal State (S_4)	+100
Move into the Wall State (S_8)	-100
All other valid moves	-1
Move into an invalid/out-of-bounds cell	-100

0.8.2 The Q-Learning Mechanism

The Q-Table stores the estimated maximum future reward, $Q(S, A)$, for taking action A in state S .

Parameters

1. **Learning Rate (α):** $\alpha = 0.5$.
2. **Discount Factor (γ):** $\gamma = 0.9$.

The Q-Learning Update Rule

The Q-value for a state-action pair is updated after every transition from S_t to S_{t+1} :

$$Q(S_t, A_t) \leftarrow (1 - \alpha)Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) \right]$$

0.8.3 Q-Table Progression

Initial Q-Table (Epoch 0)

All Q-values are initialized to zero.

	State	N	S	E	W
Initial Q-Table	S_1	0	0	0	0
	S_2	0	0	0	0
	S_3	0	0	0	0
	S_4 (Goal)	0	0	0	0
	...	...	...	...	...
	S_{13} (Start)	0	0	0	0
	...	...	...	...	...

Q-Table After First Epoch (Example values)

A single epoch is one journey from S_{13} to the S_4 goal state (path will be random). The values near the goal state (S_3) show the first signs of reward propagation.

	State	N	S	E	W
Q-Table After Epoch 1	S_1	-0.5	0	0	-0.5
	S_2	0	0	-0.5	0
	S_3	0	0	50.0	0
	S_4 (Goal)	0	0	0	0
	S_5	0	0	0	0
	S_6	0	-0.5	0	0
	S_7	0	0	-50.5	0
	S_8 (Wall)	0	0	0	0
	S_9	0	0	-0.5	-0.5
	S_{10}	0	0	0	0
	S_{11}	0	0	0	0
	S_{12}	0	0	0	0
	S_{13} (Start)	-0.5	0	0	0
	S_{14}	0	0	0	0
	S_{15}	0	0	0	0
	S_{16}	0	0	0	0

Q-Table After Tenth Epoch (Example values)

After several epochs, the positive reward has propagated backward, and negative values from the wall (S_8) have also propagated. The agent has started learning the optimal path (e.g., $S_2 \rightarrow S_3 \rightarrow S_4$). The maximum Q-value in each state indicates the optimal action.

State		N	S	E	W
S_1		-1.5	0	18.0	-1.5
S_2		-1.5	0	40.5	0
S_3		-1.5	0	89.0	0
S_4 (Goal)		0	0	0	0
S_5		18.0	0	-3.0	-1.5
S_6		40.5	0	-10.0	-3.0
S_7		89.0	0	-90.5	-10.0
Q-Table After Epoch 10	S_8 (Wall)	0	0	0	0
	S_9	-1.5	-3.0	14.5	-1.5
	S_{10}	-3.0	-10.0	32.0	-3.0
	S_{11}	-10.0	-32.0	70.0	-10.0
	S_{12}	-32.0	-70.0	-1.5	70.0
	S_{13} (Start)	14.5	-1.5	-3.0	-1.5
	S_{14}	-3.0	-10.0	25.0	-3.0
	S_{15}	-10.0	-32.0	56.5	-10.0
	S_{16}	-32.0	-70.0	-1.5	-1.5

0.9 Concept Learning, Version Space, and the Candidate Elimination Algorithm

This section follows the formulation presented in *Machine Learning* by Tom M. Mitchell (McGraw-Hill, 1997), Chapter 2.

0.9.1 1. Target Concept (c)

The **target concept** is the true, unknown Boolean-valued function that the learner aims to approximate. It represents the ground-truth rule that correctly classifies every instance.

$$c : X \rightarrow \{0, 1\} \quad \text{or equivalently} \quad c : X \rightarrow \{+, -\}$$

The objective of learning is to identify a hypothesis h such that $h(x) = c(x)$ for all $x \in X$.

2. Hypothesis (h)

A **hypothesis** is any candidate description of the target concept that can be represented by the learner. In concept learning, hypotheses are typically expressed as conjunctions of constraints over the instance attributes.

The learning task is to select a hypothesis

$$h \in H$$

that best approximates the target concept c based on observed data.

3. Hypothesis Space (H)

The **hypothesis space** H is the set of all hypotheses that the learner can possibly represent. Each hypothesis corresponds to a conjunction of constraints, one per attribute.

For each attribute A_i , the possible constraints are:

- A **specific value** $v \in V_i$
- The symbol $?$, indicating no constraint (matches any value)
- The symbol $\emptyset$, indicating no value is allowed

The symbol $\emptyset$ is used to define the most specific hypothesis.

Hypothesis Space Size: Example

Consider the attributes

$\langle \text{Sky, Temperature, Forecast} \rangle$

- Sky: {Sunny, Cloudy} ($V_1 = 2$)
- Temperature: {Warm, Cold} ($V_2 = 2$)
- Forecast: {Strong, Light} ($V_3 = 2$)

For an attribute with V_i possible values, there are $V_i + 2$ distinct constraints. Thus, the size of the hypothesis space is:

$$|H| = \prod_{i=1}^3 (V_i + 2) = (2 + 2)(2 + 2)(2 + 2) = 64$$

4. Version Space ($VS_{H,D}$)

Given a hypothesis space H and training data D , the **version space** is defined as the subset of hypotheses that are consistent with all observed examples.

$$VS_{H,D} = \{h \in H \mid h \text{ is consistent with } D\}$$

Rather than storing all hypotheses in the version space explicitly, it can be represented compactly using boundary sets.

5. Consistency

A hypothesis h is **consistent** with an example (x, y) if:

$$\text{consistent}(h, x, y) = \begin{cases} h(x) = 1, & \text{if } y = + \\ h(x) = 0, & \text{if } y = - \end{cases}$$

6. General-to-Specific Ordering

For two hypotheses $h_1, h_2 \in H$:

$$h_1 \succeq h_2 \iff \forall x \in X, (h_2(x) = 1 \Rightarrow h_1(x) = 1)$$

This relation defines a **partial order** over the hypothesis space, from most general to most specific.

7. Boundary Sets

Candidate Elimination maintains two boundary sets:

- **Specific boundary** (S): the set of maximally specific hypotheses in $VS_{H,D}$
- **General boundary** (G): the set of maximally general hypotheses in $VS_{H,D}$

Every hypothesis h in the version space satisfies:

$$\exists g \in G, s \in S \text{ such that } g \succeq h \succeq s$$

8. Candidate Elimination Algorithm

Initialization

- Initialize S to the most specific hypothesis in H
- Initialize G to the most general hypothesis in H

Update Rules

For each training example (x, y) :

- If $y = +$:
 - Remove from G any hypothesis that does not cover x
 - Minimally generalize hypotheses in S to cover x
 - Remove hypotheses in S that are more general than others
- If $y = -$:
 - Remove from S any hypothesis that covers x
 - Minimally specialize hypotheses in G to exclude x
 - Remove hypotheses in G that are more specific than others

9. Removing Redundant Hypotheses

- In S , remove s_1 if there exists s_2 such that

$$s_1 \succeq s_2 \quad \text{and} \quad s_1 \neq s_2$$
- In G , remove g_1 if there exists g_2 such that

$$g_2 \succeq g_1 \quad \text{and} \quad g_1 \neq g_2$$

10. Key Properties

- Maintains the complete version space implicitly
- Assumes noise-free training data
- Fails if the target concept is not in H
- Incrementally shrinks the hypothesis space

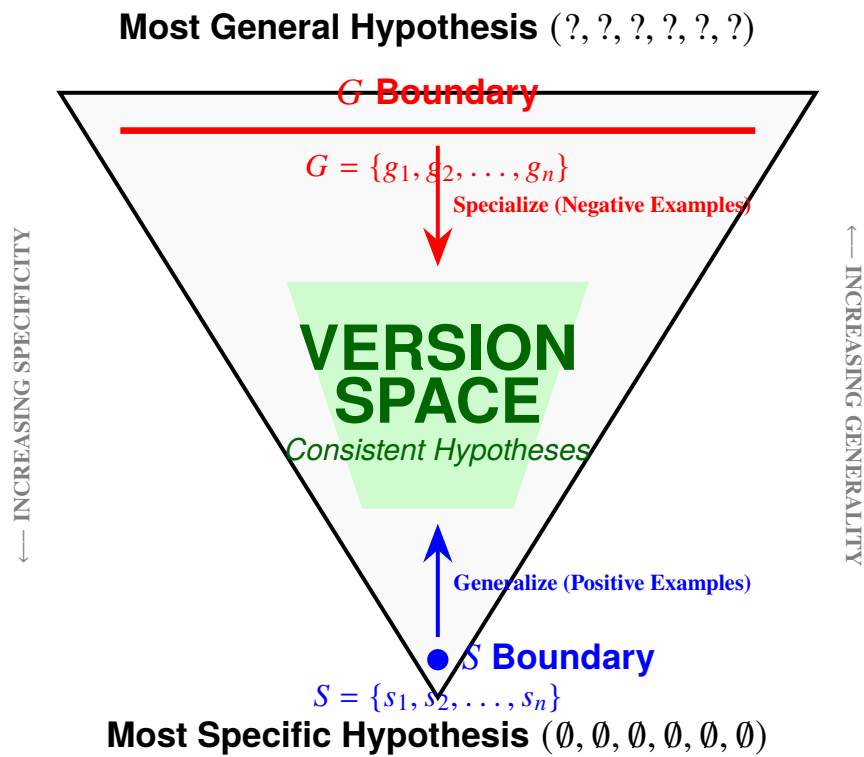
0.9.2 Worked Example: Candidate Elimination Algorithm

Attributes and Instance Space

Consider the instance space defined by the attributes:

$\langle \text{Sky, Temperature, Forecast} \rangle$

- Sky: {Sunny, Cloudy}
- Temperature: {Warm, Cold}
- Forecast: {Strong, Light}



Boundary	Training Example	Action	Goal
S (Specific)	Positive (+)	Generalize	Expand to include the new positive instance.
G (General)	Negative (-)	Specialize	Shrink/Restrict to exclude the negative instance.

Table 0.1 Boundary Modification Logic in Candidate-Elimination

Training Data

The learner observes the following sequence of labeled training examples:

- $x_1 : (+, \langle \text{Sunny, Warm, Strong} \rangle)$
- $x_2 : (+, \langle \text{Cloudy, Warm, Strong} \rangle)$
- $x_3 : (-, \langle \text{Sunny, Cold, Strong} \rangle)$
- $x_4 : (-, \langle \text{Cloudy, Warm, Light} \rangle)$
- $x_5 : (+, \langle \text{Cloudy, Cold, Strong} \rangle)$

Initialization

The boundary sets are initialized as:

$$S_0 = \{\langle \emptyset, \emptyset, \emptyset \rangle\} \quad G_0 = \{\langle ?, ?, ? \rangle\}$$

Processing Training Examples

Example 1: Positive

$$x_1 = \langle \text{Sunny}, \text{Warm}, \text{Strong} \rangle$$

- S_0 does not cover x_1 and is minimally generalized.
- G_0 covers x_1 and remains unchanged.

$$S_1 = \{\langle \text{Sunny}, \text{Warm}, \text{Strong} \rangle\}, \quad G_1 = \{\langle ?, ?, ? \rangle\}$$

Example 2: Positive

$$x_2 = \langle \text{Cloudy}, \text{Warm}, \text{Strong} \rangle$$

- S_1 is generalized on the Sky attribute.
- G_1 remains unchanged.

$$S_2 = \{\langle ?, \text{Warm}, \text{Strong} \rangle\}, \quad G_2 = \{\langle ?, ?, ? \rangle\}$$

Example 3: Negative

$$x_3 = \langle \text{Sunny}, \text{Cold}, \text{Strong} \rangle$$

- S_2 does not cover x_3 and remains unchanged.
- G_2 covers x_3 and must be minimally specialized.

Minimal specializations of G_2 that exclude x_3 and are more general than S_2 :

$$G_3 = \{\langle ?, \text{Warm}, ? \rangle\}$$

$$S_3 = S_2$$

Example 4: Negative

$$x_4 = \langle \text{Cloudy}, \text{Warm}, \text{Light} \rangle$$

- S_3 does not cover x_4 and remains unchanged.
- G_3 covers x_4 and is minimally specialized.

$$G_4 = \{\langle ?, \text{Warm}, \text{Strong} \rangle\}, \quad S_4 = S_3$$

Example 5: Positive

$$x_5 = \langle \text{Cloudy}, \text{Cold}, \text{Strong} \rangle$$

- S_4 is generalized on the Temperature attribute.
- G_4 does not cover x_5 and is removed.

$$S_5 = \{ \langle ?, ?, \text{Strong} \rangle \}$$

Since G must be at least as general as S :

$$G_5 = \{ \langle ?, ?, \text{Strong} \rangle \}$$

Final Version Space

The boundary sets converge:

$$S_{\text{final}} = G_{\text{final}} = \{ \langle ?, ?, \text{Strong} \rangle \}$$

Learned Target Concept

The target concept is satisfied if and only if the forecast is **Strong**.

Interpretation

The convergence of S and G indicates that the target concept has been uniquely identified within the hypothesis space. This behavior is guaranteed only when the data is noise-free and the target concept lies within H .

0.10 Inductive Bias

0.10.1 Definition

According to Tom Mitchell [?], an *inductive bias* is the set of assumptions that a learning algorithm uses to generalize beyond the observed training data.

Formally, given a hypothesis space $\mathcal{H}$ and training data D , a learning algorithm selects a hypothesis $h \in \mathcal{H}$. The inductive bias determines *which hypothesis is preferred* among all hypotheses consistent with D .

$$\text{Inductive Bias} = \text{Restrictions or preferences over } \mathcal{H} \quad (0.1)$$

0.10.2 Need for Inductive Bias

Mitchell states that learning without inductive bias is impossible. This follows from the *No Free Lunch Theorem*, which implies that without assumptions, no learner can generalize better than random guessing.

“A learner that makes no a priori assumptions regarding the identity of the target concept has no rational basis for classifying any unseen instances.”

0.10.3 Example: Inductive Bias in Concept Learning

Consider a binary classification task where many hypotheses fit the training data. A learner may prefer the **simplest hypothesis**, reflecting an inductive bias towards shorter or less complex representations.

$$h^* = \arg \min_{h \in \mathcal{H}} \text{Complexity}(h) \quad (0.2)$$

This preference is an instance of *Occam's Razor*, a commonly used inductive bias.

0.10.4 Formal View

Let:

- $\mathcal{X}$ be the instance space
- $\mathcal{Y}$ be the label space
- $\mathcal{H}$ be the hypothesis space

The inductive bias determines how the learner extrapolates from a finite set $D \subset \mathcal{X} \times \mathcal{Y}$ to unseen instances $x \in \mathcal{X} \setminus D$.

$$\forall x \notin D, \quad h(x) \text{ depends on inductive bias} \quad (0.3)$$

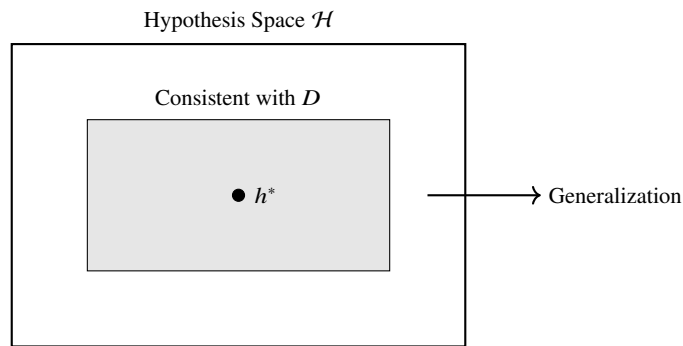


Fig. 0.3 Role of Inductive Bias in Hypothesis Selection

0.10.5 Illustrative Diagram

0.11 Types of Inductive Bias

Inductive biases can be broadly categorized into two types:

- **Restriction Bias (Language Bias):** This limits the set of hypotheses the learner considers.
 - *Example:* The Candidate-Elimination algorithm assumes the target concept is a simple conjunction of attributes.
- **Preference Bias (Search Bias):** This does not limit the hypothesis space but defines a preference for certain hypotheses over others.
 - *Example:* Occam's Razor—preferring the shortest decision tree that fits the data.

0.12 Comparison of Algorithm Biases

Algorithm	Inductive Bias
FIND-S	The target concept is in H , and all instances are negative unless proven otherwise.
Candidate-Elimination	The target concept c is contained within the hypothesis space H .
ID3 (Decision Trees)	Shorter trees are preferred over larger trees; trees with high Information Gain near the root are preferred.

Table 0.2 Inductive Biases of Standard Algorithms

0.13 Summary

Inductive bias allows a learner to perform a "jump" from observed data to unobserved instances. While a stronger bias allows for more aggressive generalization, it risks excluding the true target concept if the underlying assumptions are incorrect.

0.14 Decision Tree Induction Method

Decision Tree Induction is a recursive, greedy algorithm used to induce a classification tree from a training set. The ID3 algorithm, developed by Ross Quinlan and detailed by Tom Mitchell, uses **Information Gain** based on **Entropy** to select the best attribute at each node.

0.15 The Dataset: PlayTennis

The following table represents the training data used to determine if a day is suitable for playing tennis.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Table 0.3 The PlayTennis Training Dataset

0.16 Mathematical Foundations

0.16.1 Entropy

Entropy measures the impurity of a sample S . For a binary classification:

$$Entropy(S) = -p_{\oplus} \log_2 p_{\oplus} - p_{\ominus} \log_2 p_{\ominus}$$

where $p_{\oplus}$ is the proportion of positive examples and $p_{\ominus}$ is the proportion of negative examples.

0.16.2 Information Gain

Information Gain is the expected reduction in entropy caused by partitioning on attribute A :

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

0.17 Level-Wise Construction

0.17.1 Level 0: Root Node Computation

The total dataset S has 14 examples: 9 positive and 5 negative.

$$Entropy(S) = - \left(\frac{9}{14} \right) \log_2 \left(\frac{9}{14} \right) - \left(\frac{5}{14} \right) \log_2 \left(\frac{5}{14} \right) \approx 0.940$$

After calculating Gain for all attributes:

- $Gain(S, Outlook) = 0.246$
- $Gain(S, Humidity) = 0.151$
- $Gain(S, Wind) = 0.048$
- $Gain(S, Temperature) = 0.029$

Decision: Outlook has the highest gain and is chosen as the Root.

0.17.2 Level 1: Branching

1. **Overcast:** Contains 4 examples, all are "Yes". Entropy = 0. Create **Leaf: Yes**.
2. **Sunny:** Contains 5 examples (D1, D2, D8, D9, D11). 2 Yes, 3 No.
3. **Rain:** Contains 5 examples (D4, D5, D6, D10, D14). 3 Yes, 2 No.

0.17.3 Sub-Table: Outlook = Sunny

Day	Temp	Humidity	Wind	Play?
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

In this sub-set, $Gain(S_{sunny}, Humidity)$ is 0.970, which is the highest. **Decision:** If Humidity is High $\rightarrow$ No; If Humidity is Normal $\rightarrow$ Yes.

0.18 Decision Tree Rules

The tree can be represented as a set of IF-THEN rules:

- **IF** Outlook = Overcast **THEN** PlayTennis = Yes
- **IF** Outlook = Sunny **AND** Humidity = High **THEN** PlayTennis = No
- **IF** Outlook = Sunny **AND** Humidity = Normal **THEN** PlayTennis = Yes
- **IF** Outlook = Rain **AND** Wind = Strong **THEN** PlayTennis = No
- **IF** Outlook = Rain **AND** Wind = Weak **THEN** PlayTennis = Yes

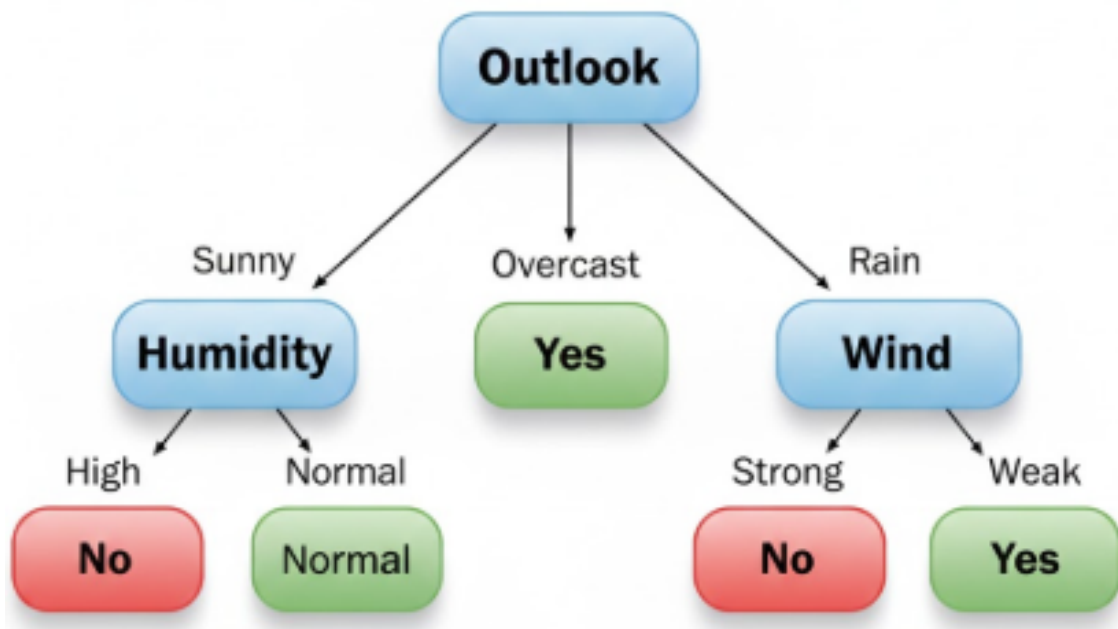


Fig. 0.4 Model Architecture

0.19 Inductive Bias

The inductive bias of ID3 is a **preference bias**:

1. Shorter trees are preferred over longer trees.
2. Trees that place high information gain attributes close to the root are preferred.

0.20 Unit-2

0.21 Artificial Neural Networks

Artificial Neural Networks (ANNs) are computational models inspired by biological neural systems. They consist of interconnected processing elements (neurons) that compute weighted sums of inputs followed by nonlinear activation functions. Neural networks provide a rich hypothesis space capable of representing complex nonlinear functions.

0.22 Biological vs Artificial Neural Networks

Although ANNs are inspired by biological neural systems, they are simplified mathematical abstractions. The key differences are summarized in Table 0.4.

Table 0.4 Comparison between Biological Neural Networks and Artificial Neural Networks

Aspect	Biological NN	Artificial NN
Basic unit	Biological neuron	Artificial neuron
Signal type	Electrochemical	Numerical values
Processing	Parallel and asynchronous	Mostly synchronous
Learning	Synaptic plasticity (Hebbian)	Gradient descent, backpropagation
Speed	Slow (ms)	Fast (ns)
Energy efficiency	Very high	Relatively low
Connectivity	Highly recurrent	Mostly feed-forward
Fault tolerance	Very high	Moderate
Interpretability	Natural	Black-box

0.23 Artificial Neuron Model

An artificial neuron computes:

$$z = \sum_{i=1}^n w_i x_i + b \quad (0.4)$$

$$y = f(z) \quad (0.5)$$

where $f(\cdot)$ is the activation function.

0.24 Activation Functions

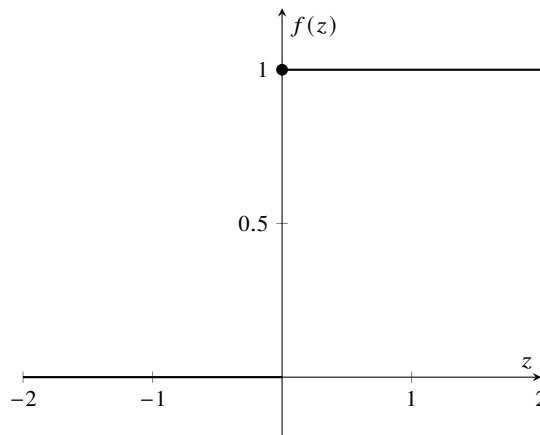
Activation functions determine whether a neuron fires and introduce nonlinearity into the network.

0.24.1 Step Function

Mathematical Definition

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases} \quad (0.6)$$

Diagram

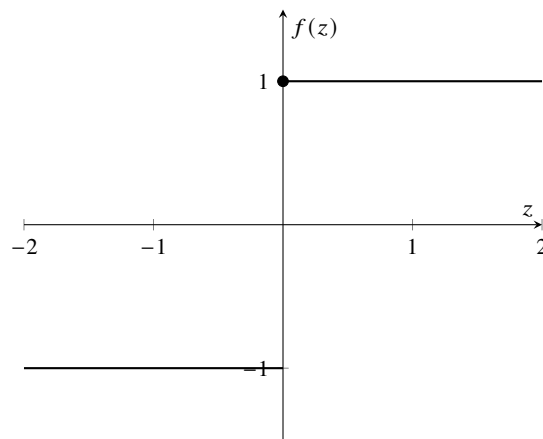


Usage: Perceptron learning **Limitation:** Not differentiable $\rightarrow$ cannot be used with backpropagation

0.24.2 Sign Function

Mathematical Definition

$$f(z) = \begin{cases} +1, & z \geq 0 \\ -1, & z < 0 \end{cases} \quad (0.7)$$

Diagram

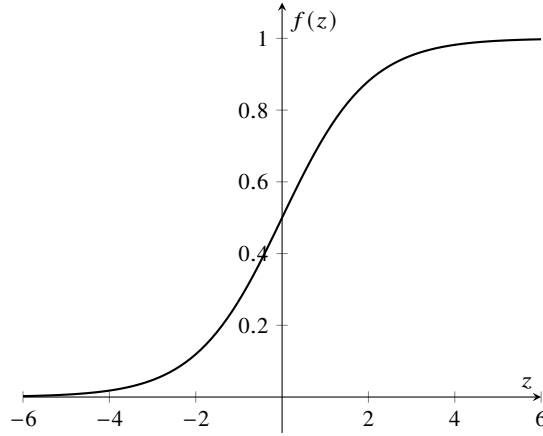
Usage: Bipolar perceptron models **Limitation:** Non-differentiable

0.24.3 Sigmoid Function**Mathematical Definition**

$$f(z) = \frac{1}{1 + e^{-z}} \quad (0.8)$$

Derivative

$$\frac{df}{dz} = f(z)(1 - f(z)) \quad (0.9)$$

Diagram

Usage: Multilayer neural networks **Advantage:** Differentiable → enables backpropagation
 Consider the multi-layer feed-forward neural network shown in Figure 8.1, consisting of:

- Input layer with M neurons (linear transfer function)
- Hidden layer with N neurons (log-sigmoid transfer function)
- Output layer with P neurons (tan-sigmoid transfer function)

Let:

- I_i denote the i^{th} input
- v_{ij} denote the weight from input neuron i to hidden neuron j
- w_{jk} denote the weight from hidden neuron j to output neuron k

0.24.4 Forward Pass Computation

Input Layer (Linear Transfer Function):

$$I_{oi} = I_i \quad (0.10)$$

Hidden Layer Input:

$$H_{Ij} = \sum_{i=1}^M v_{ij} I_{oi} \quad (0.11)$$

Hidden Layer Output (Log-Sigmoid):

$$H_{Oj} = f(H_{Ij}) = \frac{1}{1 + e^{-H_{Ij}}} \quad (0.12)$$

Output Layer Input:

$$O_{Ik} = \sum_{j=1}^N w_{jk} H_{Oj} \quad (0.13)$$

Output Layer Output (Tan-Sigmoid):

$$O_{Ok} = g(O_{Ik}) = \tanh(O_{Ik}) \quad (0.14)$$

0.24.5 Error Function

For a given training sample, the error at the output layer is defined as:

$$E = \frac{1}{2} \sum_{k=1}^P (T_k - O_{Ok})^2 \quad (0.15)$$

where T_k is the target output for neuron k .

0.24.6 Backpropagation as Chain Rule of Derivatives

Backpropagation computes the gradient of the error function with respect to each weight using the **chain rule of calculus**.

0.24.6.1 Output Layer Weight Update (w_{jk})

Using chain rule:

$$\frac{\partial E}{\partial w_{jk}} = \frac{\partial E}{\partial O_{Ok}} \cdot \frac{\partial O_{Ok}}{\partial O_{Ik}} \cdot \frac{\partial O_{Ik}}{\partial w_{jk}} \quad (0.16)$$

Compute each term:

$$\frac{\partial E}{\partial O_{Ok}} = -(T_k - O_{Ok}) \quad (0.17)$$

$$\frac{\partial O_{Ok}}{\partial O_{Ik}} = 1 - O_{Ok}^2 \quad (\text{tan-sigmoid derivative}) \quad (0.18)$$

$$\frac{\partial O_{Ik}}{\partial w_{jk}} = H_{Oj} \quad (0.19)$$

Output Layer Delta Term:

$$\delta_k = (T_k - O_{Ok})(1 - O_{Ok}^2) \quad (0.20)$$

Weight Update Rule:

$$\Delta w_{jk} = \eta \delta_k H_{Oj} \quad (0.21)$$

0.24.6.2 Hidden Layer Weight Update (v_{ij})

The error at hidden neuron j is obtained by propagating the output error backward.

Using chain rule:

$$\frac{\partial E}{\partial v_{ij}} = \sum_{k=1}^P \frac{\partial E}{\partial O_{Ik}} \cdot \frac{\partial O_{Ik}}{\partial H_{Oj}} \cdot \frac{\partial H_{Oj}}{\partial H_{Ij}} \cdot \frac{\partial H_{Ij}}{\partial v_{ij}} \quad (0.22)$$

Compute each term:

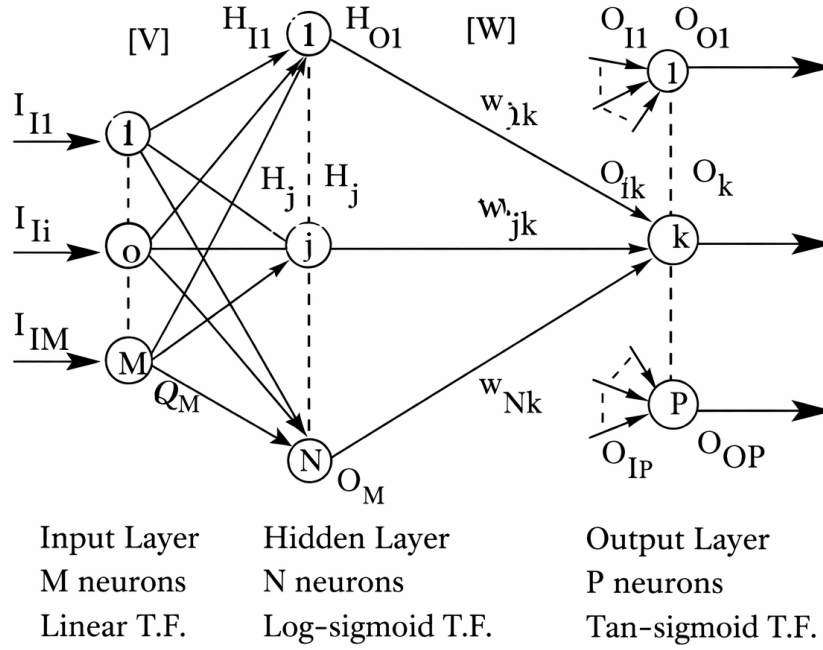


Figure 8.1: Multi-layer feed-forward neural network

Fig. 0.5 NN Model

$$\frac{\partial O_{Ik}}{\partial H_{Oj}} = w_{jk} \quad (0.23)$$

$$\frac{\partial H_{Oj}}{\partial H_{Ij}} = H_{Oj}(1 - H_{Oj}) \quad (\text{log-sigmoid derivative}) \quad (0.24)$$

$$\frac{\partial H_{Ij}}{\partial v_{ij}} = I_{oi} \quad (0.25)$$

Hidden Layer Delta Term:

$$\delta_j = H_{Oj}(1 - H_{Oj}) \sum_{k=1}^P \delta_k w_{jk} \quad (0.26)$$

Weight Update Rule:

$$\Delta v_{ij} = \eta \delta_j I_{oi} \quad (0.27)$$

0.24.7 Backpropagation Algorithm Summary

1. Initialize v_{ij} and w_{jk} randomly
2. Perform forward pass to compute H_{Oj} and O_{Ok}
3. Compute output error δ_k

4. Backpropagate error to hidden layer to compute δ_j
5. Update weights w_{jk} and v_{ij}
6. Repeat until convergence

0.25 Unit-2.1

0.26 Gradient-Based Optimization Methods

0.26.1 Problem Formulation

Let the training dataset be

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. Let $\theta \in \mathbb{R}^d$ denote the model parameter vector.

The empirical risk minimization problem is defined as:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f(x_i; \theta)), \quad (0.28)$$

where $\mathcal{L}(\cdot)$ is a differentiable loss function.

The objective is to find

$$\theta^* = \arg \min_{\theta} J(\theta).$$

0.26.2 Gradient Descent: Vector–Matrix Form

The gradient of the objective function is

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \mathcal{L}(y_i, f(x_i; \theta)). \quad (0.29)$$

The parameter update rule for batch gradient descent is

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} J(\theta^{(t)}), \quad (0.30)$$

where $\eta > 0$ is the learning rate.

Squared Error Loss Example:

For $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$ and $\hat{y} = X\theta$, the gradient becomes

$$\nabla_{\theta} J(\theta) = \frac{1}{N} X^{\top} (X\theta - y). \quad (0.31)$$

0.26.3 Stochastic Gradient Descent (SGD)

Instead of computing gradients over the full dataset, SGD approximates the gradient using a single sample:

$$\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \mathcal{L}_i(\theta). \quad (0.32)$$

The update rule is

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} \mathcal{L}_i(\theta^{(t)}). \quad (0.33)$$

The stochastic gradient is an unbiased estimator:

$$\mathbb{E}[\nabla_{\theta} \mathcal{L}_i(\theta)] = \nabla_{\theta} J(\theta). \quad (0.34)$$

0.26.4 Geometric Interpretation

Gradient descent updates parameters in the direction of the *negative gradient*, which corresponds to the direction of *steepest descent* in the parameter space.

- The gradient $\nabla_{\theta} J(\theta)$ is orthogonal to the level sets of $J(\theta)$.
- Each update moves θ toward a lower contour of the objective function.
- In ill-conditioned problems, zig-zag trajectories may occur.

SGD introduces noise into this trajectory, which can help escape shallow local minima and saddle points.

0.26.5 Momentum-Based Gradient Descent

Momentum accelerates convergence by accumulating a velocity vector.

$$v^{(t)} = \mu v^{(t-1)} + \nabla_{\theta} J(\theta^{(t)}), \quad (0.35)$$

$$\theta^{(t+1)} = \theta^{(t)} - \eta v^{(t)}, \quad (0.36)$$

where $\mu \in [0, 1)$ is the momentum coefficient.

Interpretation:

Momentum dampens oscillations and accelerates learning along consistent gradient directions.

0.26.6 RMSProp

RMSProp adapts learning rates using an exponential moving average of squared gradients.

$$s^{(t)} = \rho s^{(t-1)} + (1 - \rho)(\nabla_{\theta} J(\theta^{(t)}))^2, \quad (0.37)$$

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\eta}{\sqrt{s^{(t)}} + \epsilon} \nabla_{\theta} J(\theta^{(t)}), \quad (0.38)$$

where $\rho \in [0, 1)$ and ϵ is a small constant.

Interpretation:

RMSProp prevents aggressive decay of learning rates observed in AdaGrad.

0.26.7 Adam Optimizer

Adam combines momentum and RMSProp using first and second moment estimates.

$$m^{(t)} = \beta_1 m^{(t-1)} + (1 - \beta_1) g^{(t)}, \quad (0.39)$$

$$v^{(t)} = \beta_2 v^{(t-1)} + (1 - \beta_2) (g^{(t)})^2, \quad (0.40)$$

where $g^{(t)} = \nabla_{\theta} J(\theta^{(t)})$.

Bias-corrected estimates:

$$\hat{m}^{(t)} = \frac{m^{(t)}}{1 - \beta_1^t}, \quad (0.41)$$

$$\hat{v}^{(t)} = \frac{v^{(t)}}{1 - \beta_2^t}. \quad (0.42)$$

Final update rule:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{\eta}{\sqrt{\hat{v}^{(t)}} + \epsilon} \hat{m}^{(t)}. \quad (0.43)$$

0.26.8 Comparison of Optimization Methods

Method	Learning Rate	Adaptivity	Noise	Memory
Gradient Descent	Fixed	No	None	Low
SGD	Fixed/Decay	No	High	Low
Momentum	Fixed	No	Moderate	Low
RMSProp	Adaptive	Yes	Moderate	Medium
Adam	Adaptive	Yes	Moderate	Medium

0.26.9 Summary

Gradient-based optimization methods differ primarily in how gradients are estimated and how learning rates are controlled. While classical gradient descent provides stable convergence, modern adaptive methods such as Adam offer faster convergence and robustness for large-scale and deep learning problems.